

Examen Programación 1º DAW 12/03/2024

Ejer 1.- Debemos crear GestAirport software para la gestión de aeropuertos, el programa controlará la información de salida de los vuelos.

Tendremos un menú con:

1. Mostrar cola de vuelos ordenada.
2. Añadir un nuevo vuelo a la cola.
3. Eliminar un vuelo de la cola.
4. Dar salida a un vuelo.
5. Actualizar lista de vuelos.
6. Modificar horarios de un vuelo.
7. Salir

1. Mostrar cola de vuelos ordenada: Debe mostrar por pantalla la información de todos los vuelos que salgan en las próximas 24 horas. (Para ello es obligatorio usar alguna de las implementaciones del método sort de la clase Collections)
2. Añadir un nuevo vuelo a la cola: se pedirán los datos necesarios para crear un nuevo vuelo y se añadirá a la lista en lugar que le corresponda por orden de salida. Si el vuelo tiene una hora de salida anterior a la fecha/hora actual se debe lanzar una excepción llamada **FechaHoraSalidaNoValidaException**, esta excepción debe ser gestionada en el main informando al usuario del error y continúe con el menú principal .
3. Eliminar un vuelo de la cola: se pedirá el código del vuelo a eliminar y se borrará de la cola.
4. Dar salida a un vuelo: eliminar de la cola al primer vuelo de la lista.
5. Actualizar lista de vuelos: Comprobar la hora actual y eliminar de la lista aquellos vuelos cuya hora de salida ya haya sido superada.
6. Modificar horarios de un vuelo: pedir la cantidad de minutos que se debe retrasar el vuelo y actualizar los valores de salida y llegada del vuelo. Si el usuario intenta poner un valor negativo se debe lanzar una excepción **ValorNoValidoException**, informando al usuario y solicitando un nuevo valor positivo.
7. Salir: finalizar la ejecución del programa.

Para la realización y pruebas se adjunta una librería que debéis utilizar como base, sino se utiliza se entenderá el programa como erróneo. La carga de datos debe realizarse automáticamente al inicio de la ejecución del programa, convirtiendo el array bidimensional proporcionado por la clase **Datos** en objetos de tipo **Vuelo** (esta clase debe de estar relacionada con **VueloAbstract**)

Para validar el funcionamiento se utilizarán los datos de de prueba suministrados.

Para el almacenamiento de los vuelos podéis utilizar la estructura de almacenamiento que creáis conveniente.

El main debe contener el mínimo de líneas de código posible y estará incluido en la clase **GestAirport (nombre del proyecto)**.

Para el recorrido de los elementos de la colección o mapa que uséis, es obligatorio, usar como mínimo en una ocasión la clase **Iterator**.

El código debe seguir los paradigmas de la POO, por lo que toda la gestión de los datos se realizará en una clase llamada **Control**.

Entrega: un archivo .zip en el que se incluya el proyecto completo en formato Netbeans. Dicho proyecto, debe ser exportable a otras instancias de Netbeans, sin necesidad de modificar ningún archivo, para poder realizar la corrección. Si la estructura de archivos no es correcta o se requieren modificaciones para poder ejecutar el código, se descontará un 20 % de la nota final del examen.